SOFTWARE ENGINEERING PROJECT REPORT

Tool Selected:

SonarQube — Static Code Analysis & Quality Inspection

Course: Software Engineering
Instructor: Ms. Shazmeen Batool
Submission Date: 27 April 2026

Team Members

S.No	Name	Roll Number
1	Abdul Hayy Khan	24F-AI-051
2	Abdullah Farooq	24F-AI-050
3	Muhammad Fasih	24F-AI-016

Dawood University of Engineering & Technology
Department of Artificial Intelligence

1. Introduction

Software Engineering is not just about writing code — it is about writing code that works correctly, stays secure, and can be easily understood and changed by other developers in the future. As software projects grow bigger and more complex, it becomes harder to manually check every line of code for mistakes. This is where automated tools become essential.

This project report focuses on SonarQube, a widely used open-source platform that automatically inspects source code and reports problems. Instead of a human reviewer reading through hundreds of lines of code, SonarQube does the job instantly, finding bugs, security risks, and poor-quality code patterns in seconds.

To demonstrate how SonarQube works in a real setting, our team — we applied it to our own project: ResumRank-AI, an AI-powered resume screening web application built using Python (Flask) and NLP libraries. The analysis revealed real issues in our codebase, making this exercise both educational and practically valuable.

This report documents the tool, its relevance to Software Engineering practices, the step-by-step process we followed to run the analysis, and the screenshots captured at each stage of the demonstration.

---

2. Description of the Tool — SonarQube

SonarQube is an enterprise-grade, open-source code quality platform developed by SonarSource. It performs what is called Static Code Analysis — meaning it reads and inspects source code without actually running the program. This is different from testing, where you run the code and check if it works. Static analysis finds problems in the structure of the code itself.

SonarQube supports over 30 programming languages including Python, JavaScript, Java, C++, and more. It is designed to plug directly into the Software Development Life Cycle (SDLC) and can be connected to CI/CD pipelines such as GitHub Actions, Jenkins, and GitLab CI so that code is automatically checked every time a developer pushes new changes.

Key Features of SonarQube

- **Bug Detection:** SonarQube identifies logic errors and null pointer exceptions that could cause the application to crash or behave incorrectly during real usage.
- **Vulnerability Detection:** It scans for known security weaknesses such as SQL injection risks, unsafe file uploads, and hardcoded passwords.
- **Security Hotspots:** Code that is not necessarily broken but could become a security problem is flagged for manual human review.

- **Code Smells:** These are structural problems — functions that are too long, too complex, or poorly named — that make code harder to understand and maintain.
- **Technical Debt:** SonarQube estimates how many hours a developer would need to fix all the detected code smells, giving project managers a measurable picture of code health.
- **Quality Gates:** A configurable pass/fail system. If code does not meet the set quality threshold, the Quality Gate fails and the code should not be deployed.
- **Web Dashboard:** A clean browser-based interface at localhost:9000 shows all findings visually with severity levels, file locations, and line numbers.

Figure 2.1 — SonarQube Community Edition logo and interface overview

3. About the Analyzed Project — ResumRank-AI

Project Overview

ResumRank-AI is an intelligent resume screening web application built by Abdul Hayy Khan as part of the Code Pulse Ramadan Hackathon 2026 organized by CIS Community at NEDUET. The project is publicly available on GitHub at: https://github.com/abdulhayykhan/resumrank-ai

The application solves a real-world problem: when a company posts a job opening, they may receive hundreds of resumes. Reading each one manually is slow and inconsistent. ResumRank-AI automates this process — a recruiter uploads multiple PDF resumes along with a job description, and the AI ranks the candidates from most to least relevant.

Technology Stack

- **Backend Framework:** Python (Flask) — a lightweight web framework that handles routing, file uploads, and API responses.
- **PDF Parsing:** pdfplumber — extracts raw text content from uploaded PDF resume files.
- **NLP Processing:** spaCy — a Natural Language Processing library used to extract skills, keywords, and relevant entities from resume text.
- **Resume Ranking:** Pandas — used to process, compare, and rank extracted resume data against the job description.

- **Frontend:** Vanilla JavaScript, HTML5, and CSS3 — a simple, clean browser interface with no framework dependencies.
- **Deployment Target:** Railway (Python web service hosting platform).

Why ResumRank-AI is a Good Test Case for SonarQube

ResumRank-AI is an excellent candidate for static analysis for several specific reasons:

- **File Upload Handling:** The app accepts PDF files from unknown users on the internet. Unsafe file handling is a serious security risk and exactly the kind of issue SonarQube is designed to detect.
- **NLP Pipeline Complexity:** The skill extraction and ranking logic inside ranker.py and skill_extractor.py involves multiple nested conditions and branching logic, making it likely to have high Cognitive Complexity scores.
- **Real Codebase:** Unlike textbook examples, this is production-intended code with actual logic, making the SonarQube findings genuine and meaningful.
- **Multi-file Python Project:** The codebase spans multiple modules (app.py, ranker.py, pdf_parser.py, skill_extractor.py), giving SonarQube a realistic amount of code to analyze.

---

4. Impact on Software Engineering Practices

SonarQube directly supports and automates several core pillars of Software Engineering. Below is how each practice is addressed:

4.1 — Automated Testing & Verification

Traditional software testing (unit tests, integration tests) checks whether code produces the correct output when run. SonarQube adds a complementary layer: it checks whether the code is written correctly before it is even executed. It catches unhandled exceptions, unreachable code paths, and logic flaws that may not surface in basic test runs. For ResumRank-AI, this means problems in the PDF parsing and ranking pipeline can be identified without running any resumes through the system.

4.2 — Project Management & Maintainability

SonarQube introduces the concept of Technical Debt — a measurable estimate (in developer-hours) of how much effort it would take to fix every code quality issue. This gives project managers and team leads a concrete number to work with when planning sprints and allocating time for refactoring. Instead of vague statements like 'the code needs cleaning,' a team can say 'we have 3.5 hours of technical debt to address this sprint.'

4.3 — Security & Quality Assurance

The Quality Gate system enforces a minimum acceptable standard before code can be considered ready for deployment. For our application, which handles user-uploaded files, this is critical. SonarQube flagged the file upload handling in app.py as a Security Hotspot, warning that uploaded files are not being validated for type or content — a real vulnerability that could allow malicious file uploads. Without SonarQube, this issue might only be discovered after a security incident.

4.4 — Code Reviews & Collaboration

SonarQube's issue tracking with file names, line numbers, severity ratings, and plain-English explanations makes code review discussions precise. Instead of a reviewer saying 'this function is too complicated,' they can point to a specific SonarQube finding: 'ranker.py line 47 has a Cognitive Complexity of 22, exceeding the threshold of 15.' This specificity makes team discussions faster and more actionable.

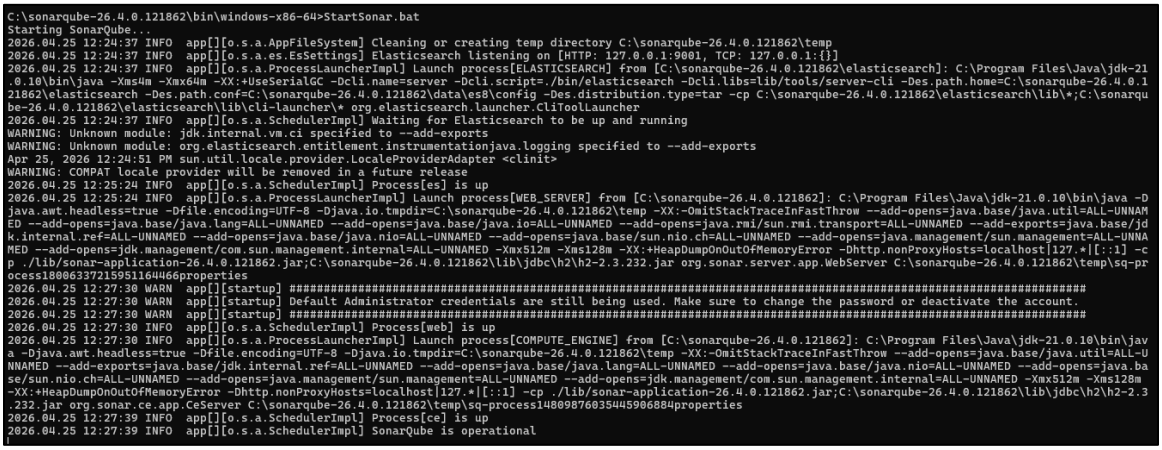
---

5. Implementation Steps & Demonstration Walkthrough

The following steps document the complete process our team followed to set up SonarQube locally and run a full static analysis of the ResumRank-AI codebase. Each step is accompanied by a screenshot taken during the actual demonstration.

Step 1: Environment Initialization — Downloading & Starting SonarQube

SonarQube Community Edition was downloaded from the official SonarSource website. Java 17 or higher is required as SonarQube runs on the Java Virtual Machine (JVM). After extracting the downloaded archive, the server was started by running **`StartSonar.bat`** from the **`bin/windows-x86-64/`** directory. After about 60–90 seconds, the SonarQube dashboard became accessible in the browser at **`http://localhost:9000`**. Default login credentials (admin/admin) were used, and a new password was set as prompted.

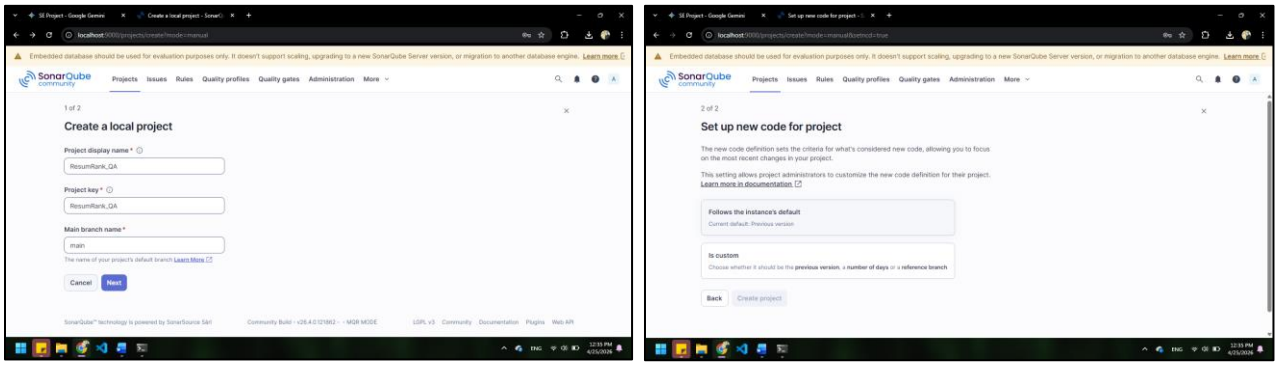


```
C:\sonarqube-26.4.0.121862\bin\windows-x86-64>StartSonar.bat
Starting SonarQube...
2026.04.25 12:24:37 INFO  app[][o.s.a.AppFileSystem] Cleaning or creating temp directory C:\sonarqube-26.4.0.121862\temp
2026.04.25 12:24:37 INFO  app[][o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:{}]
2026.04.25 12:24:37 INFO  app[][o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [C:\sonarqube-26.4.0.121862\elasticsearch]: C:\Program Files\Java\jdk-21
.0.10\bin\java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.script=./bin/elasticsearch -Dcli.libs=lib/tools/server-cli -Des.path.home=C:\sonarqube-26.4.0.1
21862\elasticsearch -Des.path.conf=C:\sonarqube-26.4.0.121862\data\es8\config -Des.distribution.type=tar -cp C:\sonarqube-26.4.0.121862\elasticsearch\lib\*;C:\sonarqu
be-26.4.0.121862\elasticsearch\lib\cli-launcher\* org.elasticsearch.launcher.CliToolLauncher
2026.04.25 12:24:37 INFO  app[][o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
WARNING: Unknown module: jdk.internal.vm.ci specified to --add-exports
WARNING: Unknown module: org.elasticsearch.entitlement.instrumentationjava.logging specified to --add-exports
Apr 25, 2026 12:24:51 PM sun.util.locale.provider.LocaleProviderAdapter <clinit>
WARNING: COMPAT locale provider will be removed in a future release
2026.04.25 12:25:24 INFO  app[][o.s.a.SchedulerImpl] Process[es] is up
2026.04.25 12:25:24 INFO  app[][o.s.a.ProcessLauncherImpl] Launch process[WEB_SERVER] from [C:\sonarqube-26.4.0.121862]: C:\Program Files\Java\jdk-21.0.10\bin\java -D
java.awt.headless=true -Dfile.encoding=UTF-8 -Djava.io.tmpdir=C:\sonarqube-26.4.0.121862\temp -XX:-OmitStackTraceInFastThrow --add-opens=java.base/java.util=ALL-UNNAM
ED --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.rmi/sun.rmi.transport=ALL-UNNAMED --add-exports=java.base/jd
k.internal.ref=ALL-UNNAMED --add-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.base/sun.nio.ch=ALL-UNNAMED --add-opens=java.management/sun.management=ALL-UNNA
MED --add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED -Xmx512m -Xms128m -XX:+HeapDumpOnOutOfMemoryError -Dhttp.nonProxyHosts=localhost|127.*|[::1] -c
p ./lib/sonar-application-26.4.0.121862.jar;C:\sonarqube-26.4.0.121862\lib\jdbc\h2\h2-2.3.232.jar org.sonar.server.app.WebServer C:\sonarqube-26.4.0.121862\temp\sq-pr
ocess18006337215951164466properties
2026.04.25 12:27:30 WARN  app[][startup] ##########################################################################################################
2026.04.25 12:27:30 WARN  app[][startup] Default Administrator credentials are still being used. Make sure to change the password or deactivate the account.
2026.04.25 12:27:30 WARN  app[][startup] ##########################################################################################################
2026.04.25 12:27:30 INFO  app[][o.s.a.SchedulerImpl] Process[web] is up
2026.04.25 12:27:30 INFO  app[][o.s.a.ProcessLauncherImpl] Launch process[COMPUTE_ENGINE] from [C:\sonarqube-26.4.0.121862]: C:\Program Files\Java\jdk-21.0.10\bin\jav
a -Djava.awt.headless=true -Dfile.encoding=UTF-8 -Djava.io.tmpdir=C:\sonarqube-26.4.0.121862\temp -XX:-OmitStackTraceInFastThrow --add-opens=java.base/java.util=ALL-U
NNAMED --add-exports=java.base/jdk.internal.ref=ALL-UNNAMED --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.ba
se/sun.nio.ch=ALL-UNNAMED --add-opens=java.management/sun.management=ALL-UNNAMED --add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED -Xmx512m -Xms128m
-XX:+HeapDumpOnOutOfMemoryError -Dhttp.nonProxyHosts=localhost|127.*|[::1] -cp ./lib/sonar-application-26.4.0.121862.jar;C:\sonarqube-26.4.0.121862\lib\jdbc\h2\h2-2.3
.232.jar org.sonar.ce.app.CeServer C:\sonarqube-26.4.0.121862\temp\sq-process1480987603544590688 4properties
2026.04.25 12:27:39 INFO  app[][o.s.a.SchedulerImpl] Process[ce] is up
2026.04.25 12:27:39 INFO  app[][o.s.a.SchedulerImpl] SonarQube is operational
```

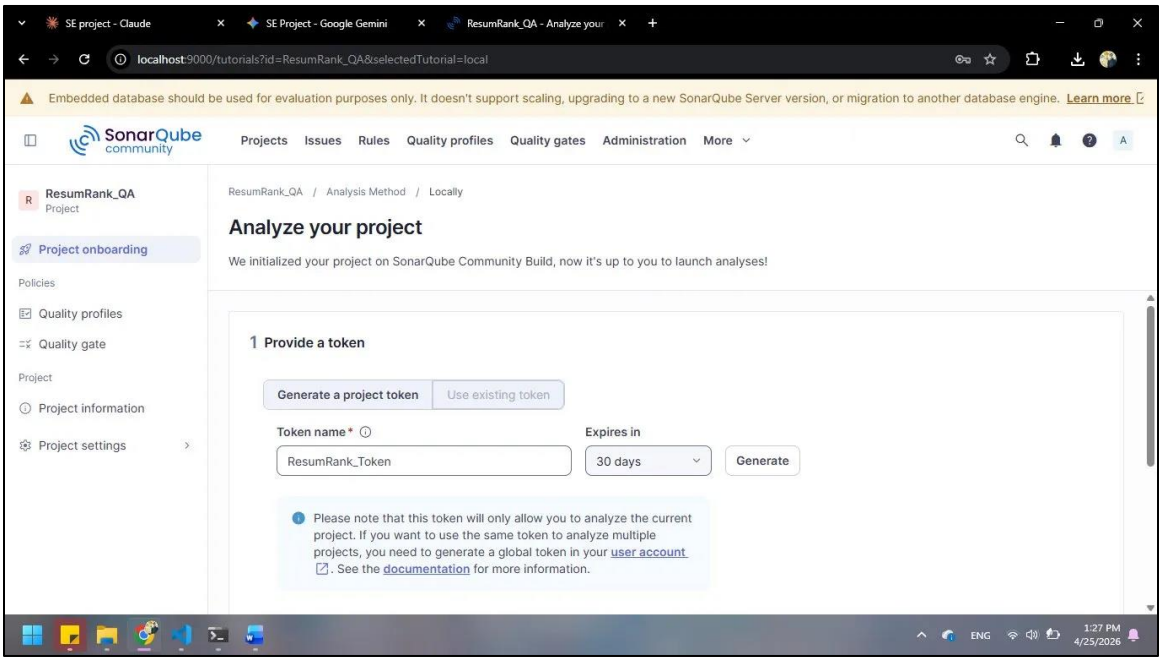
Screenshot 1 — Terminal output from StartSonar.bat confirming the SonarQube server started successfully; the final line "SonarQube is operational" indicates the server is ready and accessible at localhost:9000

Step 2: Project Creation & Authentication Token

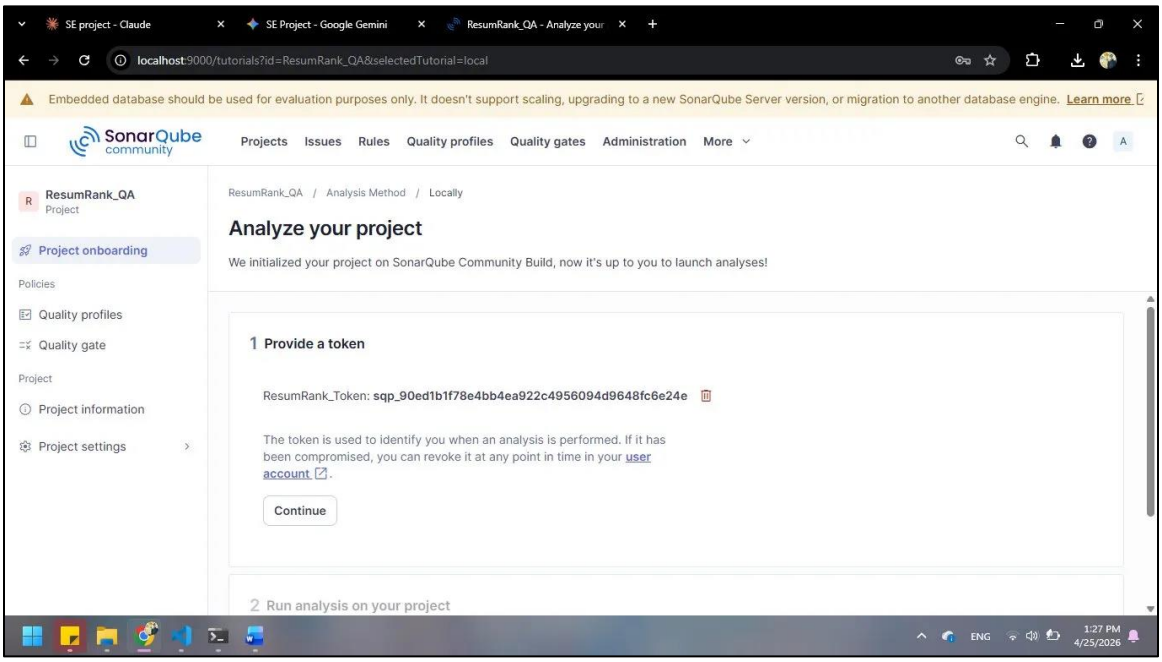
Inside the SonarQube dashboard, a new project was created using the **`Create Project`** wizard. The project was named **`ResumRank_QA`** and the analysis method was set to **`Local`**. An authentication token named **`ResumRank_Token`** was generated. This token acts as a secure key that allows the SonarScanner CLI tool (running on our machine) to communicate with the SonarQube server and upload analysis results.



Screenshot 2 — SonarQube project creation form showing the ResumRank_QA project being configured (left) alongside the Quality Gate setup screen (right)



Screenshot 3 — Token generation screen showing ResumRank_Token being configured with a 30-day expiry

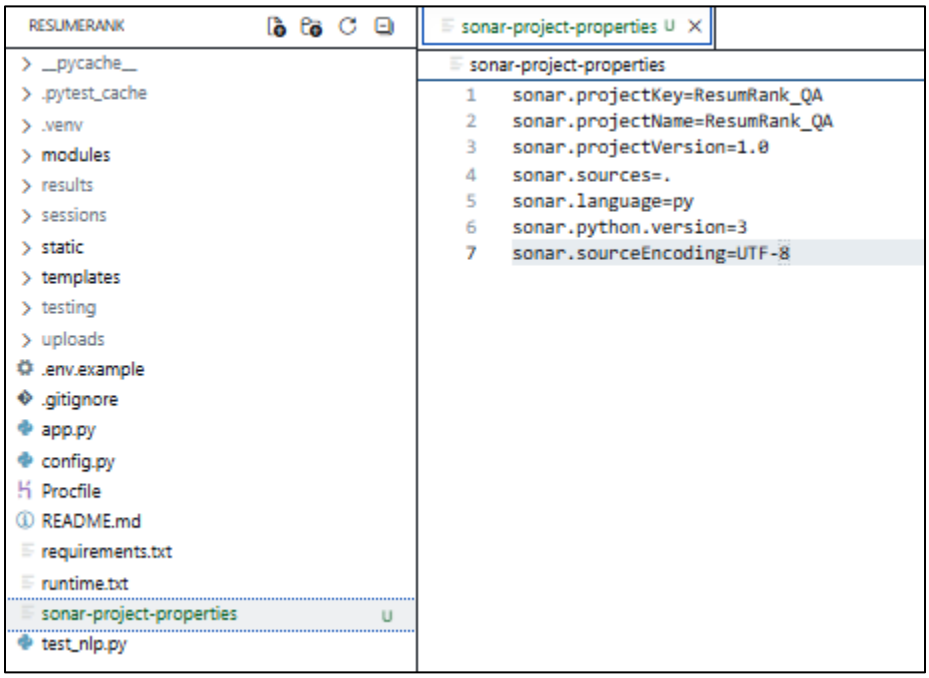


Screenshot 4 — Generated authentication token displayed; this token is used to authenticate the SonarScanner CLI

Step 3: Scanner Configuration — sonar-project.properties

A configuration file named **`sonar-project.properties`** was created in the root directory of the ResumRank-AI project. This file tells the SonarScanner exactly which project to map to on the server, which files to scan, and what programming language is being used. The configuration used was:

```
sonar.projectKey=ResumRank_QA
sonar.projectName=ResumRank_QA
sonar.projectVersion=1.0
sonar.sources=.
sonar.language=py
sonar.python.version=3
sonar.sourceEncoding=UTF-8
```



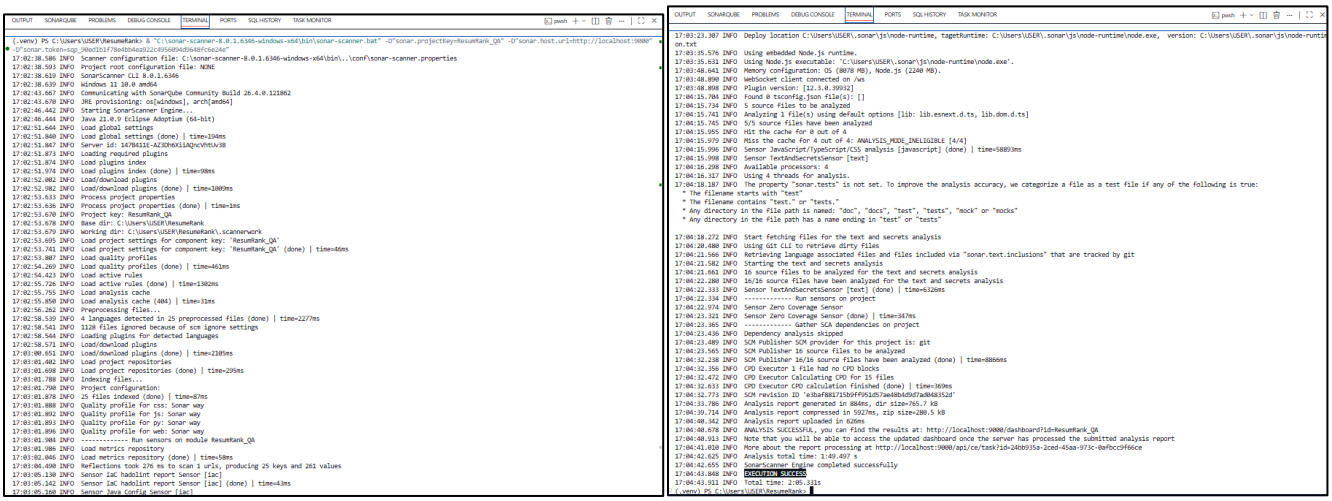
Screenshot 5 — VS Code showing the sonar-project.properties file open in the ResumRank root folder with all 7 configuration lines visible

Step 4: Running the SonarScanner CLI

The SonarScanner CLI tool was downloaded separately from the SonarSource documentation and extracted to **`C:\sonar-scanner-8.0.1.6346-windows-x86-64\`**. With the ResumRank-AI project folder open in the VS Code terminal, the following command was executed to trigger the analysis:

```
C:\sonar-scanner-8.0.1.6346-windows-x86-64\bin\sonar-scanner.bat
"-Dsonar.token=ResumRank_Token"
```

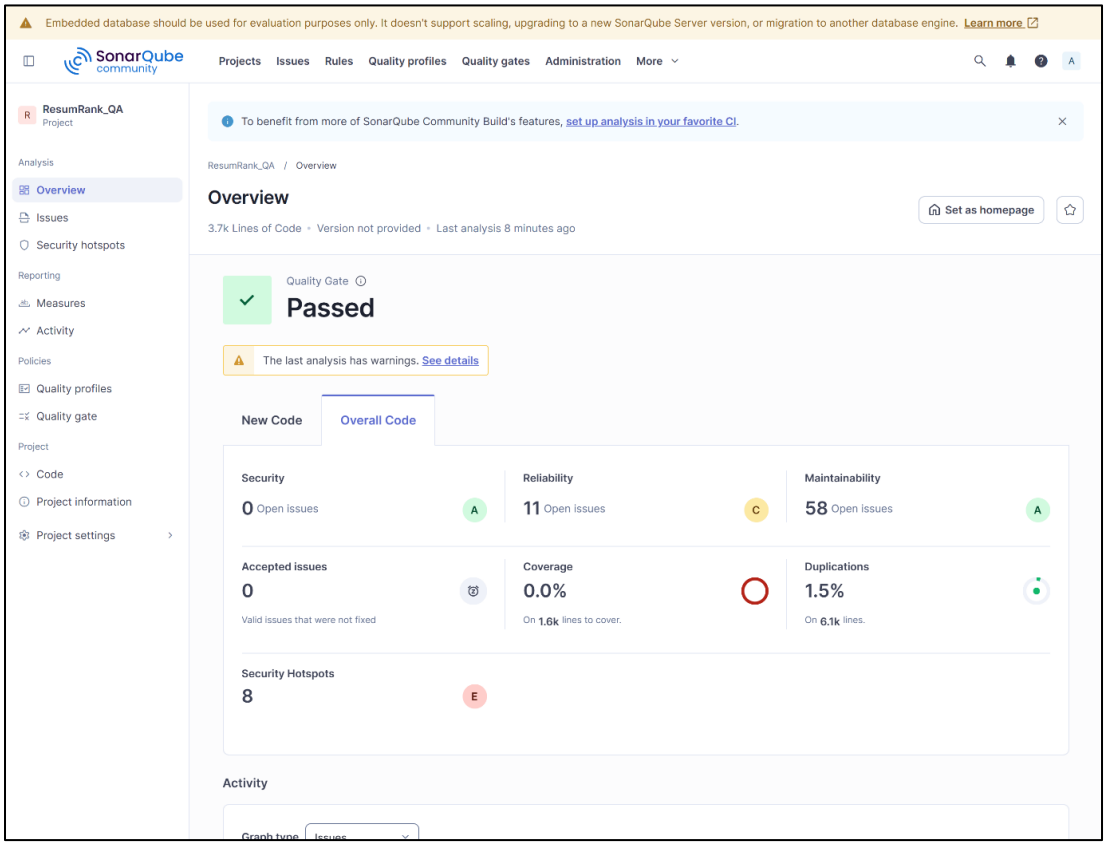
The scanner read the sonar-project.properties file, traversed all Python, HTML, and JavaScript files in the repository, and performed static analysis on each file. The terminal printed each file being processed in real time. The successful completion of the scan was confirmed by the appearance of full analysis results on the SonarQube dashboard at localhost:9000, showing 60 issues, 8 security hotspots, and a Quality Gate status of Passed.



Screenshot 6 — VS Code terminal showing the SonarScanner CLI running against the ResumRank-AI codebase

Step 5: Dashboard Review & Quality Gate

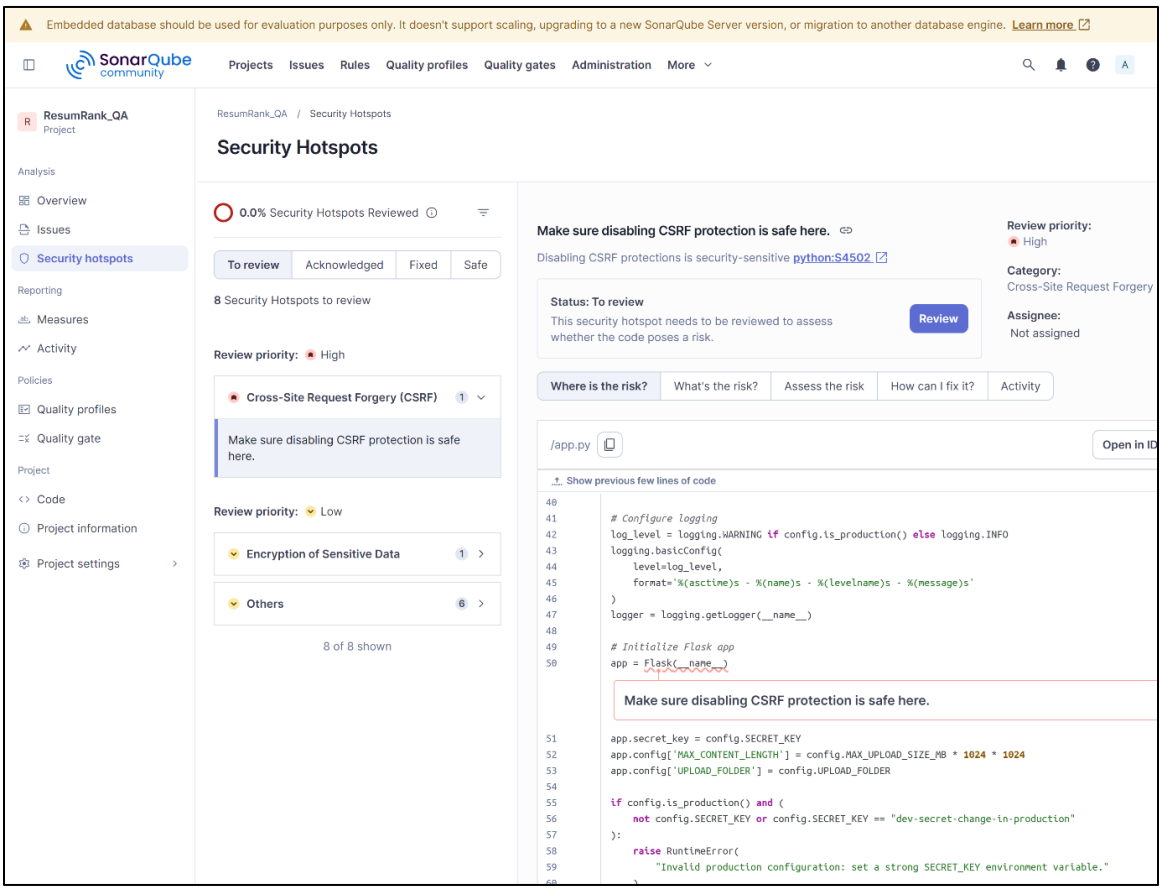
After the scan completed, the SonarQube project dashboard was refreshed to view the full analysis results. The most important indicator is the Quality Gate — a pass/fail verdict that tells whether the codebase meets the minimum quality standard set for deployment. The dashboard also showed a breakdown of Bugs, Vulnerabilities, Security Hotspots, Code Smells, and an estimate of Technical Debt.



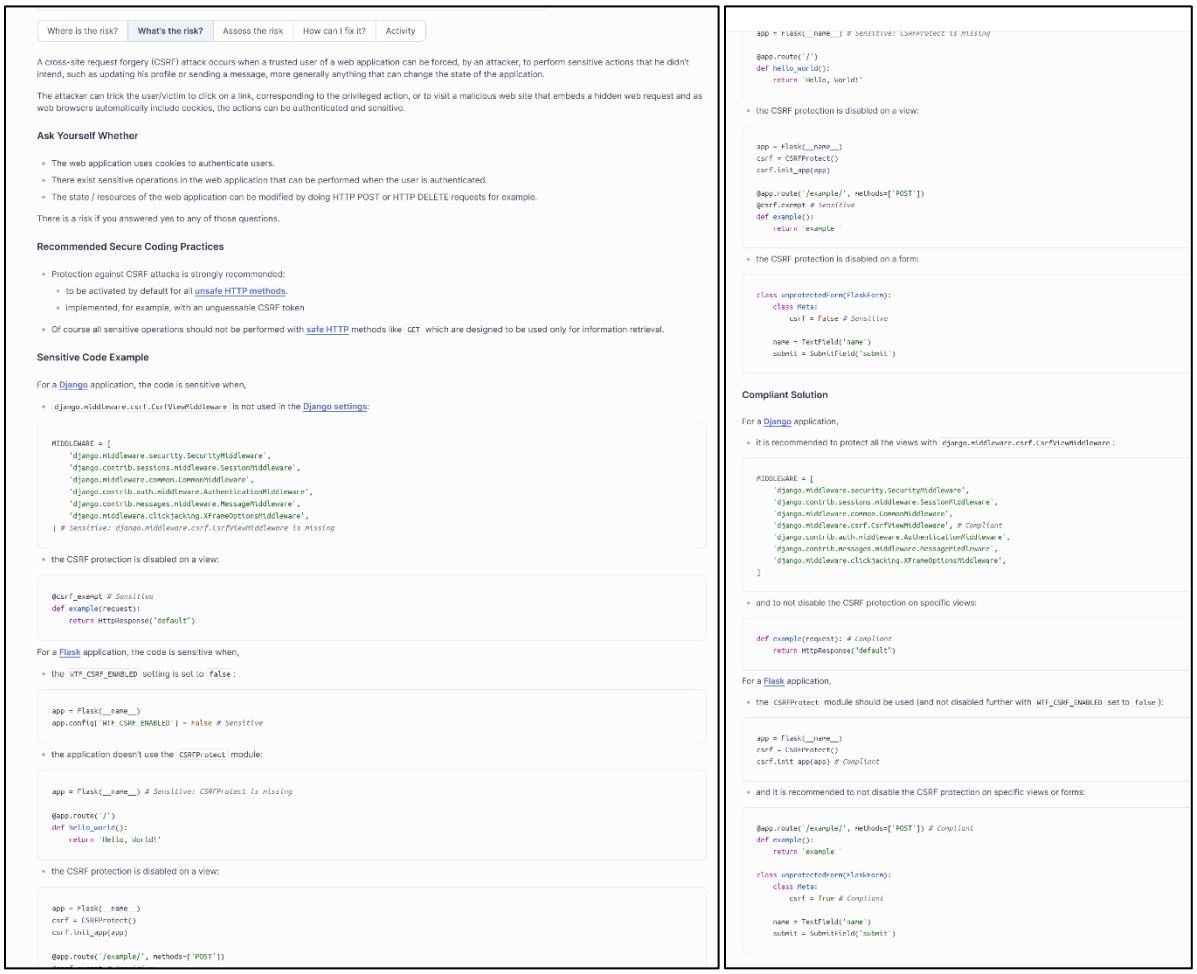
Screenshot 7 — SonarQube main project dashboard for ResumRank_QA showing the Quality Gate result and issue summary

Step 6: Deep Dive — Security Hotspots

The Security Hotspots tab was opened to examine code patterns flagged as potentially unsafe. Since ResumRank-AI accepts PDF uploads from users, the file handling logic in **`app.py`** and **`pdf_parser.py`** was flagged. Each hotspot entry shows the exact file, line number, risk category, and a plain-English explanation. Clicking into a hotspot displays the flagged code lines alongside SonarQube's recommended fix — for example, validating the file type and size before processing.



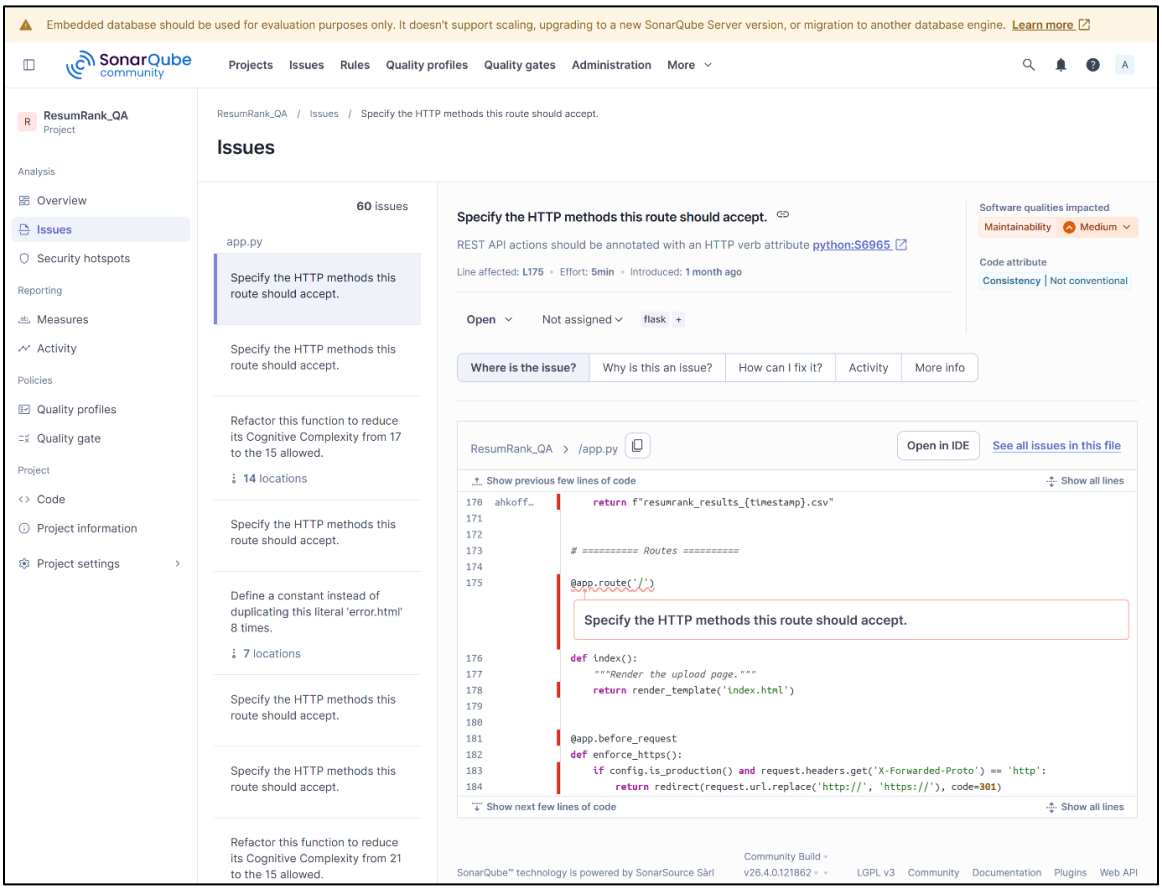
Screenshot 8 — Security Hotspots tab showing flagged file-handling code with SonarQube's risk explanation



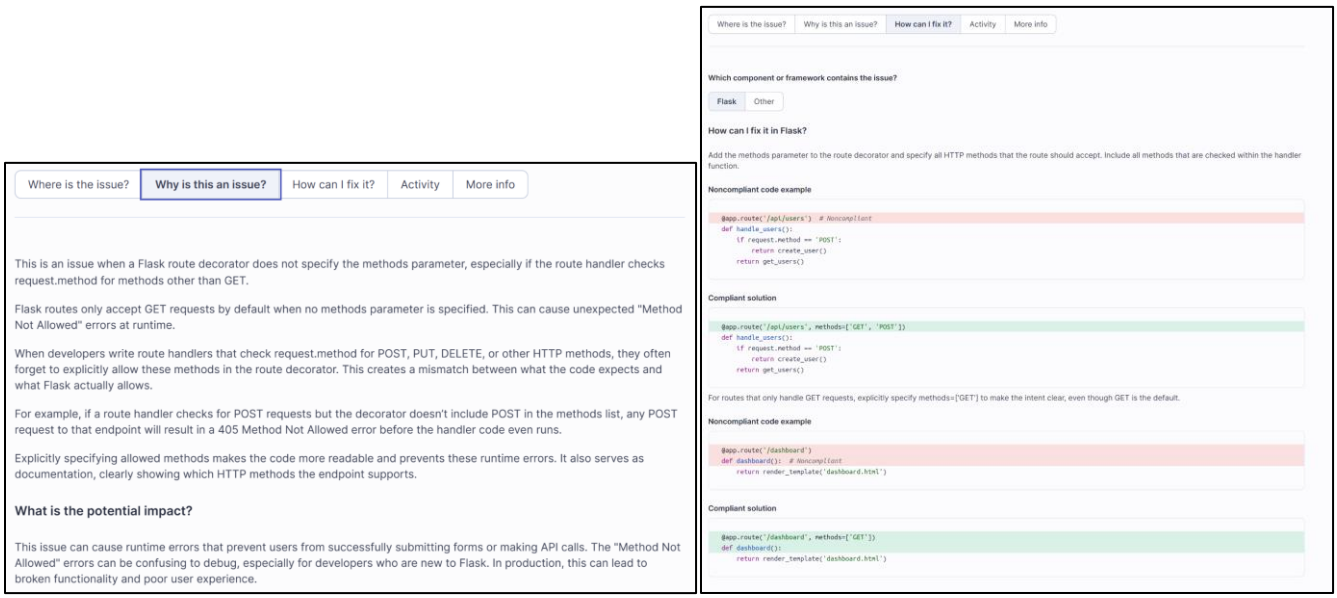
Screenshot 9 — Expanded security hotspot showing the specific flagged line in the backend and the recommended remediation

Step 7: Deep Dive — Code Smells & Technical Debt

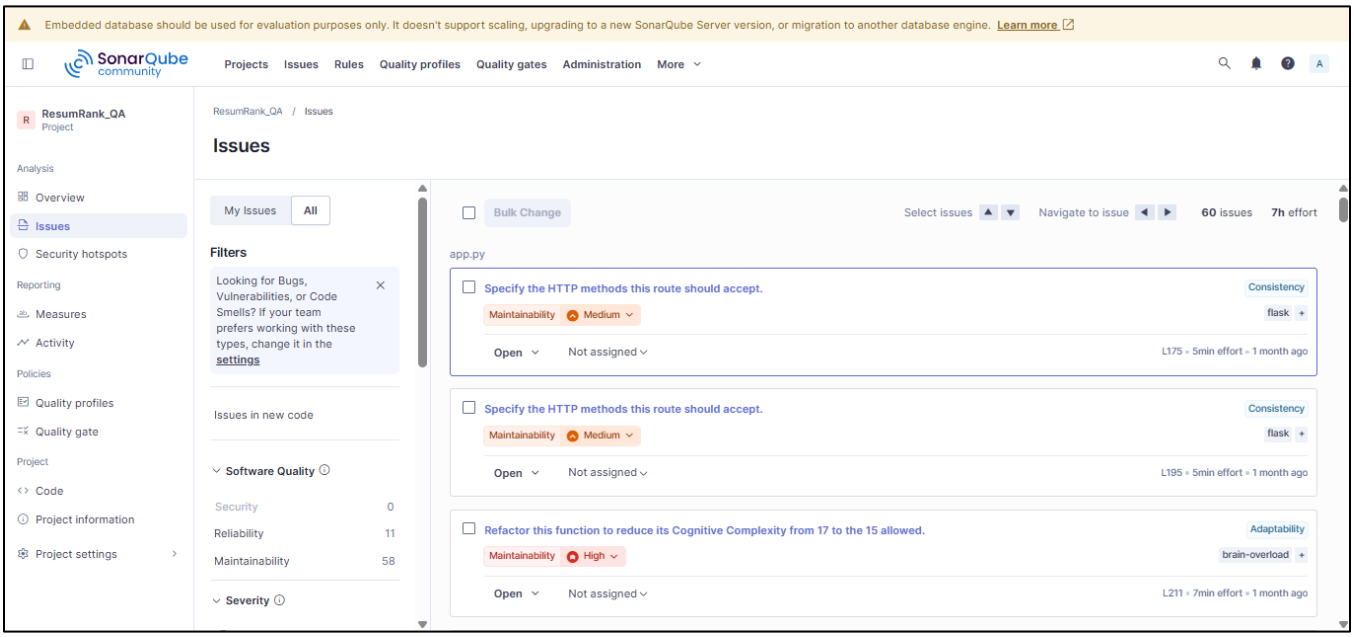
The Issues tab was filtered to show Code Smells only. The most prominent finding was high Cognitive Complexity in **`ranker.py`** and **`skill_extractor.py`**. Cognitive Complexity is a SonarQube metric that counts branching paths, nested conditions, and logical operators in a function. Functions scoring above 15 are flagged. A high score means the function is hard to read, test, and maintain. SonarQube displayed the exact line, score, and a suggestion to break the function into smaller, simpler pieces.



Screenshot 10 — Issues tab filtered to Code Smells showing cognitive complexity findings with line numbers and severity



Screenshot 11 — Expanded code smell view showing the specific function with high complexity and SonarQube's suggested fix



Screenshot 12 — Overview of all detected code smells with estimated technical debt in developer-hours

6. Conclusion

This project successfully demonstrated the practical application of SonarQube — a professional-grade static code analysis tool — on a real, self-built Python web application. By running SonarQube on ResumRank-AI, our team gained hands-on exposure to how automated quality inspection fits into the modern Software Development Life Cycle.

The analysis produced real, actionable findings: security hotspots in the file upload handling logic that could expose the application to malicious inputs, and high cognitive complexity in the ranking and skill extraction modules that would make future maintenance difficult. These are not hypothetical textbook problems — they are genuine issues in code we wrote ourselves.

SonarQube proves that code quality is not something that can be fully guaranteed by human review alone, especially as projects grow. By integrating tools like SonarQube into the development workflow, software teams can catch problems earlier, quantify technical debt, enforce quality standards through Quality Gates, and ultimately deliver software that is more secure, reliable, and maintainable.

For a field like Artificial Intelligence where Python-based applications are increasingly being deployed in production environments that handle sensitive user data, static code analysis is not optional — it is an essential engineering discipline.